Canopy Compiler Error System — Deep Audit & Redesign Plan

Date: 2026-02-24 **Scope**: Full error pipeline across all compiler phases **Goal**: Make Canopy the best-in-class compiler error experience

---

PART 1 — CURRENT STATE AUDIT

1.1 Architecture Overview

```
SOURCE CODE (.can file)
         │
         ▼
┌──────────────────────────────────────────────────────────┐
│ COMPILER PHASES (each produces phase-specific errors)    │
│                                                          │
│  Parse ──► Syntax.Error       (6,708 lines, 100+ ctors) │
│  Canon ──► Canonicalize.Error (1,208 lines, 42 ctors)   │
│  Type  ──► Type.Error         (1,906 lines, 3 ctors)    │
│  Pattern ► Pattern.Error      (150 lines,  2 ctors)     │
│  Import ─► Import.Error       (168 lines,  4 ctors)     │
│  Main  ──► Main.Error         (108 lines,  3 ctors)     │
│  Docs  ──► Docs.Error         (219 lines,  5 ctors)     │
│  Optimize ► (NONE)            ← CRITICAL GAP            │
│  Generate ► (NONE)            ← CRITICAL GAP            │
└──────────────────────────┬───────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────┐
│ ERROR AGGREGATION (Reporting.Error)                      │
│                                                          │
│  data Error                                              │
│    = BadSyntax    Syntax.Error                           │
│    | BadImports   (NE.List Import.Error)                 │
│    | BadNames     (OneOrMore Canonicalize.Error)         │
│    | BadTypes     Localizer (NE.List Type.Error)         │
│    | BadMains     Localizer (OneOrMore Main.Error)       │
│    | BadPatterns  (NE.List Pattern.Error)                │
│    | BadDocs      Docs.Error                             │
└──────────────────────────┬───────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────┐
│ REPORT CONVERSION (toReports)                            │
│                                                          │
│  data Report = Report                                    │
│    { _title   :: String                                  │
│    , _region  :: A.Region                                │
│    , _sgstns  :: [String]    ← ALWAYS EMPTY (dead field) │
```

```
│    , _message :: D.Doc                                    │
│    }                                                      │
└───────────────────────┬──────────────────────────────────┘
                        │
                        ▼
┌──────────────────────────────────────────────────────────┐
│ DOCUMENT RENDERING                                        │
│                                                           │
│  Code.toSnippet  → source snippet with line numbers       │
│  Code.toPair     → two related locations (max 2)          │
│  D.Doc           → ANSI colored pretty-printing           │
│  D.encode        → JSON with style info                   │
└───────────────────────┬──────────────────────────────────┘
                        │
                        ▼
┌──────────────────────────────────────────────────────────┐
│ OUTPUT                                                    │
│                                                           │
│  Terminal  → ANSI colors to stderr                        │
│  JSON     → structured { path, name, problems }           │
│  Plain   → stripped colors for piping                     │
└──────────────────────────────────────────────────────────┘
```

1.2 Phase-by-Phase Analysis

PARSING (`Reporting.Error.Syntax` — 6,708 lines)

Error types: ~100 constructors across 20+ sub-ADTs (`Module` , `Decl` , `Expr` , `Pattern` , `Type` , `If` , `Case` , `Let` , `Record` , `Tuple` , `List` , `Func` , `Char` , `String` , `Number` , `Space` , `Escape` , `Exposing`).

Data captured at error site:

- Row/Col position (always)
- Nested context via `specialize` combinator wrapping inner errors with outer constructors
- Source byte offset tracked in parser state

Context propagation:

```haskell
-- Parse/Primitives.hs:284-289
specialize :: (x -> Row -> Col -> y) -> Parser x a -> Parser y a
specialize addContext (Parser parser) =
  Parser $ \state@(State _ _ _ _ row col) cok eok cerr eerr ->
    let cerr' r c tx = cerr row col (addContext (tx r c))
        eerr' r c tx = eerr row col (addContext (tx r c))
     in parser state cok eok cerr' eerr'
```

Rendering quality: HIGH. Uses conversational tone, shows code snippets, provides specific suggestions.

```
-- MODULE NAME MISMATCH ---------------------------------------- src/Main.can

It looks like this module name is out of sync:
```

```
3| module App exposing (..)
          ^^^
I need it to match the file path, so I was expecting to see `Main` here.

    App -> Main
```

Problems identified:

#	Issue	Severity
P1	**Parser stops on first error** — no recovery, no multi-error	HIGH
P2	`specialize` **overwrites inner region** — outer context row/col replaces inner, losing precise error location for deeply nested failures	HIGH
P3	**Some messages are stringly-typed** — `"I ran into an unexpected " ++ term` scattered throughout the 6,700-line file	MEDIUM
P4	**Indentation errors don't report actual/expected levels** — "Try adding some spaces" without specifying how many	MEDIUM
P5	**Error title capitalization inconsistent** — mix of "MODULE NAME MISSING", "STRAY PAREN", "EXPECTING MODULE NAME"	LOW

CANONICALIZATION (`Reporting.Error.Canonicalize` — 1,208 lines)

Error types: 42 constructors covering name resolution, duplicate definitions, recursive types, exports, imports, FFI, ports, shadowing.

Data captured at error site:

- `A.Region` for all errors (good)
- `PossibleNames { _locals :: Set Name, _quals :: Map Name (Set Name) }` for not-found errors (enables "did you mean")
- Module origins for ambiguity errors (`ModuleName.Canonical`)
- Duplicate locations (`A.Region` pair)

Suggestion system: Uses `Reporting.Suggest.sort` (Damerau-Levenshtein edit distance) to rank up to 4 similar names. Also has hard-coded JavaScript-to-Canopy operator mappings (`===` → `==` , `!=` → `/=` , `**` → `^`).

Rendering quality: HIGH. Context-aware messages with specific suggestions.

Problems identified:

#	Issue	Severity
C1	`RecursiveDecl` **stores cycle names without regions** — `[Name.Name]` instead of `[(Name.Name, A.Region)]`, user can't find cycle nodes	HIGH
C2	**FFI errors are stringly-typed** — `FFIParseError A.Region FilePath String` uses raw `String` for parse error, `FFIInvalidType` uses raw `String` for type error description	HIGH

C3	**Import errors lack import-site location** — `ImportExposingNotFound` doesn't show where the import statement is	MEDIUM
C4	**Operator errors use hard-coded if/else chains** — `NotFoundBinop` has `if op == "===" then ... else if op == "!="` etc. mixing data with rendering	MEDIUM
C5	**No cascade prevention** — if an import fails, all uses of that import's values appear as "not found" errors, flooding the user	HIGH
C6	**`PossibleNames` doesn't pre-rank suggestions** — ranking computed at render time, not available for JSON/LSP output	LOW

TYPE INFERENCE (`Reporting.Error.Type` — 1,906 lines, `Type.Error` — 491 lines)

Error types: 3 constructors — `BadExpr` , `BadPattern` , `InfiniteType` — but with rich context via `Expected` / `Category` / `Context` / `SubContext` sub-ADTs (12 `Context` variants, 16 `Category` variants, 5 `PContext` variants).

Data captured at error site:

- `A.Region` — expression/pattern location
- `Category` / `PCategory` — what kind of value failed
- `T.Type` (actual and expected) — full error-type representation for diff rendering
- `Expected tipe` — captures WHY the type was expected (`NoExpectation` , `FromContext` , `FromAnnotation`)

Type diff system (`Type.Error.hs`): Sophisticated structural diff that identifies specific `Problem` types:

- `IntFloat` , `StringFromInt` , `StringFromFloat` , `AnythingToBool` , `AnythingFromMaybe`
- `ArityMismatch Int Int`
- `BadFlexSuper Direction Super Name Type`
- `BadRigidVar Name Type`
- `FieldTypo Name [Name]` , `FieldsMissing [Name]`

Rendering quality: EXCELLENT. Context-sensitive messages per `Context` variant, type comparison with colored diffs, problem-specific hints.

Problems identified:

#	Issue	Severity
T1	**Only first `Problem` gets a hint** — `problemsToHint (problem : _) = problemToHint problem`, multiple problems → only one hint shown	HIGH
T2	**No blame assignment** — can't distinguish "your annotation is wrong" from "your implementation is wrong"	HIGH
T3	**Type variable names show no provenance** — `a` could be from annotation, inference, or constraint; user can't tell which	MEDIUM
T4	**Field typo suggestions are string-distance only** — doesn't consider type compatibility for smarter suggestions	MEDIUM

T5	**No "did you forget to import?" suggestion** — when type is unknown, doesn't check if it exists in unimported modules	HIGH
T6	**Localizer can produce overly qualified names** — `Elm.Html.Html msg` instead of `Html msg` when import exists	LOW

PATTERN MATCHING (`Reporting.Error.Pattern` + `Nitpick.PatternMatches` — 16,035 lines)

Error types: 2 constructors — `Incomplete` (missing cases) and `Redundant` (unreachable pattern).

Rendering quality: GOOD. Shows unhandled pattern examples, links to custom-types documentation.

Problems: Minimal — this is a mature, well-tested subsystem (Maranget's algorithm).

OPTIMIZATION (`Optimize/*` — ~50,000 bytes across 7 files)

Error types: **NONE.**

Current state: All optimization functions return `Names.Tracker Opt.Expr`. No `Either`, no error ADT. Impossible states would crash via partial functions or `error`.

Build system fallback: `CompileOptimizeError FilePath String` — a raw `String` with no structure.

#	Issue	Severity
O1	**No structured error types exist**	CRITICAL
O2	**Impossible states crash instead of reporting**	CRITICAL
O3	**Build system uses stringly-typed wrapper**	HIGH

CODE GENERATION (`Generate/*` — ~35,000 bytes across 5 files)

Error types: **NONE.**

Current state: All generation functions produce `Builder` output directly. No error representation.

Build system fallback: `MakeBadGenerate String` with the message "This is likely a compiler bug. Please report it."

#	Issue	Severity
G1	**No structured error types exist**	CRITICAL
G2	**FFI linkage errors at generation time have no representation**	HIGH
G3	**All errors dismissed as "compiler bug"** — even user-caused ones	HIGH

1.3 Cross-Cutting Issues

Report Type is Anemic

```
data Report = Report
  { _title   :: String      -- ← raw String, not typed
  , _region  :: A.Region    -- ← single region, no multi-span
  , _sgstns  :: [String]    -- ← DEAD FIELD: always []
  , _message :: D.Doc       -- ← opaque rendering, can't be post-processed
  }
```

The `_sgstns` field is populated in exactly 2 of ~200 error constructors. It's effectively dead code.

JSON Output Missing Critical Data

```
reportToJson (Report.Report title region _sgstns message) =
  E.object
    [ "title" ==> E.chars title
    , "region" ==> encodeRegion region
    , "message" ==> D.encode message     -- ← rendered Doc, not structured data
    ]
```

Missing from JSON: suggestions, related spans, error code, severity, fix actions.

Multi-Span Limited to 2

`Code.toPair` handles exactly 2 spans. Rust-style multi-span diagnostics with 3+ related locations are impossible.

No Error Codes

No error has a stable identifier. Can't do `canopy explain E0308`. Can't link to documentation. Can't suppress specific errors.

Suggestion System is Minimal

`Reporting.Suggest` is 38 lines: edit-distance sort, nothing more.

- No type-aware suggestions
- No import-aware suggestions
- No "common mistake" catalog
- No suggestion ranking beyond string similarity

No Warning Integration

Warnings (`UnusedImport` , `UnusedVariable` , `MissingTypeAnnotation`) are collected separately, displayed after all errors, with different styling. No unified diagnostic stream.

Hardcoded Terminal Width

Separator bars and message bars assume 80-character terminal width. No detection of actual terminal size.

1.4 Test Coverage of Error System

Area	Unit Tests	Golden Tests	Integration Tests
Syntax error rendering	0	0	0

Canonicalize error rendering	0	0	0
Type error rendering	0	0	0
Pattern error rendering	0	0	0
Import error rendering	0	0	0
Suggestion quality	0	0	0
JSON error output	0	0	0
Multi-error presentation	0	0	0
Terminal error formatting	~130 lines	0	0
Terminal error suggestions	~142 lines	0	0
Total error message tests	**~270 lines**	**0**	**0**

The 8 core error modules (~10,800 lines) have **zero** test coverage for rendered output. This means error quality regressions are undetectable.

PART 2 — BENCHMARK AGAINST BEST-IN-CLASS

2.1 Elm Compiler (Canopy's upstream)

Canopy inherits Elm's error design. Where Elm excels:

- Conversational, friendly tone ("I ran into something unexpected")
- Code snippet with line numbers and carets
- Context-specific hints (if-branch mismatch → "all branches must match")
- Documentation links ("Read to learn more")

Where Elm (and therefore Canopy) falls short:

- Single error on parse failure — no recovery
- No error codes or extended explanations
- No multi-span diagnostics
- No machine-readable suggestion format
- No LSP-compatible diagnostic structure
- Limited to 2-span display via `toPair`

2.2 Rust Compiler

Rust's diagnostic system exceeds Canopy in every structural dimension:

Feature	Rust	Canopy	Gap
Multi-span with labels	Up to N spans, each with label text	Max 2 spans, no labels	LARGE
Error codes	`E0308` with `--explain` deep docs	None	LARGE
Structured suggestions	`Suggestion { span, text, applicability }`	Dead `_sgstns` field	LARGE

Severity levels	error / warning / note / help	error only (warnings separate)	MEDIUM
Machine-readable output	`--error-format=json` with full structure	JSON with rendered Doc blob	LARGE
Lint system	`#[allow(...)]`, `#[deny(...)]`	None	MEDIUM
Related diagnostics	"note: X defined here", "help: consider Y"	Inline in message text	MEDIUM
Error recovery	Continues after errors, shows multiple	Parse: stop at first. Type: multiple.	MEDIUM

2.3 Scala Error Indexing

Scala 3 (`dotc`) assigns every error a numeric code, and the compiler website hosts `https://docs.scala-lang.org/scala3/reference/error-codes/E001.html` pages. Each error page explains:

- What the error means
- Common causes
- How to fix it
- Related errors

Canopy has no error code system and no extended documentation.

2.4 Gap Summary

```
CANOPY TODAY                          TARGET STATE
──────────────────────────────────────────────────────────

Single region per error         →    Multi-span with labels
No error codes                  →    Stable IDs + explain
Dead suggestion field           →    Structured fix actions
Rendered Doc in JSON            →    Full semantic JSON
Parse stops at first error      →    Error recovery
No cascade prevention           →    Root-cause analysis
80-char hardcoded width         →    Terminal-aware layout
0% error message test coverage  →    Golden tests for all
No optimize/generate errors     →    Full phase coverage
Suggestions = edit distance     →    Type/import/context aware
```

PART 3 — DESIGN PRINCIPLES (TARGET STATE)

Principle 1: Structure Over Strings

Every error MUST be representable as structured data. No stringly-typed error content anywhere in the pipeline.

Concrete rule: If you can't `deriving (Eq)` on an error type without hitting `String` or `Doc`, it's wrong.

Principle 2: Multi-Layered Diagnostics

Every error supports four layers:

```
Layer 1: Title      "TYPE MISMATCH"                          (always shown)
Layer 2: Summary    "The 2nd argument to `map` is not..."    (always shown)
Layer 3: Detail     Code snippet + type comparison + hint    (always shown)
Layer 4: Extended   Full tutorial via `canopy explain E0308`  (on demand)
```

Principle 3: Suggestions Are First-Class

Suggestions are NOT prose embedded in `D.Doc`. They are structured data:

```
data Suggestion = Suggestion
  { _sugSpan        :: !Region          -- where to apply
  , _sugReplacement :: !Text            -- what to insert
  , _sugMessage     :: !Text            -- human explanation
  , _sugConfidence  :: !Confidence      -- how sure we are
  }

data Confidence = Definite | Likely | Possible
```

An IDE can auto-apply `Definite` suggestions. A terminal renderer can show `Likely` as "Try this:".

Principle 4: Perfect Location UX

- Primary span: where the error IS
- Secondary spans: related locations (definition site, annotation site, first branch)
- Each span has a label explaining its role
- Code snippets show ACTUAL source (not pretty-printed AST)

Principle 5: Consistency Rules

Aspect	Convention
Title	ALL CAPS, 2-4 words: "TYPE MISMATCH", "NAMING ERROR"
Tone	Second person, present tense: "This expression is...", "I cannot find..."
Structure	Title → Summary → Snippet → Explanation → Suggestion → Note
Type names	Use import-qualified shortest form: `Html msg` not `Elm.Html.Html msg`
Ordinals	"1st", "2nd", "3rd" (human-friendly)
Lists	Oxford comma for inline, bullet list for >3 items
Colors	Red = error location, Yellow = related, Green = suggestion, Cyan = code reference

Principle 6: Error Codes & Knowledge Base

Every error gets a stable code: `E` + 4 digits.

```
E0100-E0199: Parse errors
E0200-E0299: Import/module errors
E0300-E0399: Name resolution errors
E0400-E0499: Type errors
E0500-E0599: Pattern errors
E0600-E0699: Optimization errors
E0700-E0799: Code generation errors
E0800-E0899: Build/package errors
E0900-E0999: Documentation errors
```

`canopy explain E0401` prints the full tutorial for that error.

PART 4 — ARCHITECTURE DESIGN

4.1 Core Diagnostic Type

Replace `Report` with a rich `Diagnostic`:

```haskell
-- | A compiler diagnostic with full structured information.
--
-- This is the universal error representation across all compiler phases.
-- It carries enough information for terminal rendering, JSON output,
-- LSP integration, and IDE quick-fixes.
data Diagnostic = Diagnostic
  { _diagCode       :: !ErrorCode         -- E0401
  , _diagSeverity   :: !Severity          -- Error | Warning | Info
  , _diagTitle      :: !Text              -- "TYPE MISMATCH"
  , _diagSummary    :: !Text              -- one-line plain text
  , _diagPrimary    :: !LabeledSpan       -- main error location + label
  , _diagSecondary  :: ![LabeledSpan]     -- related locations
  , _diagMessage    :: !Doc               -- full rendered explanation
  , _diagSuggestions :: ![Suggestion]     -- structured fix actions
  , _diagNotes      :: ![Text]            -- additional context
  , _diagPhase      :: !Phase             -- which compiler phase
  }

data LabeledSpan = LabeledSpan
  { _spanRegion :: !Region
  , _spanLabel  :: !Text                  -- "expected type", "defined here"
  , _spanStyle  :: !SpanStyle
  }

data SpanStyle = Primary | Secondary | Note

data Severity = Error | Warning | Info

newtype ErrorCode = ErrorCode Word16

data Suggestion = Suggestion
  { _sugSpan        :: !Region
  , _sugReplacement :: !Text
```

```
  , _sugMessage     :: !Text
  , _sugConfidence  :: !Confidence
  }

data Confidence = Definite | Likely | Possible
```

4.2 Phase-Specific Error Types (Unchanged)

Keep the existing per-phase error ADTs (`Syntax.Error` , `Canonicalize.Error` , `Type.Error` , etc.). They are well-designed. The change is in the **conversion layer**: instead of `toReport :: Source -> Error -> Report` , each phase implements `toDiagnostic :: Source -> Error -> Diagnostic` .

Add the missing phases:

```
-- NEW: Reporting.Error.Optimize
data Error
  = InvalidCycle Region Name [Name]
  | ConstantFoldFailure Region Text
  | InternalError Region Text          -- compiler bug, not user error

-- NEW: Reporting.Error.Generate
data Error
  = FFILinkageFailure Region FilePath Name
  | JSKeywordCollision Region Name
  | InternalError Region Text
```

4.3 Error Aggregation (Updated)

```
data Error
  = BadSyntax Syntax.Error
  | BadImports (NE.List Import.Error)
  | BadNames (OneOrMore Canonicalize.Error)
  | BadTypes Localizer (NE.List Type.Error)
  | BadMains Localizer (OneOrMore Main.Error)
  | BadPatterns (NE.List Pattern.Error)
  | BadDocs (NE.List Docs.Error)        -- ← fix: was single, now list
  | BadOptimize Optimize.Error          -- ← NEW
  | BadGenerate Generate.Error          -- ← NEW
```

4.4 Suggestion Engine Architecture

```
SUGGESTION SOURCES
──────────────────────────────────────────
1. Edit distance    → "Did you mean `fooBar`?"
2. Import catalog   → "Try adding `import List`"
3. Type catalog     → "Use `String.fromInt` to convert"
4. Common mistakes  → JavaScript operators, truthiness
5. Annotation       → "Add a type annotation here"
6. Pattern coverage → "Handle the `Nothing` case"
```

```
SUGGESTION PIPELINE
────────────────────────────────────────

Error ADT
  → matchSuggestionPatterns :: Error -> [SuggestionCandidate]
  → rankSuggestions :: [SuggestionCandidate] -> [Suggestion]
  → attachToDiagnostic :: [Suggestion] -> Diagnostic -> Diagnostic
```

The suggestion engine is a **separate module** (`Reporting.Suggest.Engine`) that pattern-matches on error types and produces structured suggestions. This separates suggestion logic from error rendering.

4.5 Rendering Architecture

```
                    Diagnostic
                        │
              ┌─────────┼─────────┐
              ▼         ▼         ▼
          Terminal     JSON       LSP
          Renderer   Renderer   Adapter
              │         │         │
              ▼         ▼         ▼
          ANSI Doc   JSON Value  LSP Diagnostic
```

Terminal renderer: Takes `Diagnostic` → renders multi-span code snippets with labels, colored by span style, followed by explanation and suggestions. Detects terminal width.

JSON renderer: Takes `Diagnostic` → produces:

```
{
  "code": "E0401",
  "severity": "error",
  "title": "TYPE MISMATCH",
  "summary": "The 2nd argument to `map` has type String but Float is expected",
  "primary": {
    "start": { "line": 5, "column": 20 },
    "end": { "line": 5, "column": 35 },
    "label": "this is String"
  },
  "secondary": [
    {
      "start": { "line": 3, "column": 1 },
      "end": { "line": 3, "column": 30 },
      "label": "annotation says Float"
    }
  ],
  "suggestions": [
    {
      "span": { "start": { "line": 5, "column": 20 }, "end": { "line": 5, "column":
35 } },
      "replacement": "String.toFloat input",
      "message": "Convert String to Float with String.toFloat",
      "confidence": "likely"
```

```
    }
  ],
  "message": ["The 2nd argument to ", { "bold": true, "string": "map" }, " is not
what I expect..."]
}
```

LSP adapter: Maps `Diagnostic` directly to LSP `Diagnostic` protocol type with `CodeAction` for suggestions.

PART 5 — IMPLEMENTATION PLAN

Phase 0: Foundation (Week 1)

Goal: New types, no behavior change.

1. Create `Reporting/Diagnostic.hs` with `Diagnostic`, `LabeledSpan`, `Suggestion`, `ErrorCode`, `Severity`, `Confidence` types
2. Create `Reporting/ErrorCode.hs` with error code registry (initially empty, filled per phase)
3. Create `Reporting/Suggest/Engine.hs` with suggestion pipeline stub
4. Update `Reporting/Render/Code.hs` to support multi-span rendering (generalize `toPair` to N spans)
5. Add terminal width detection in `Reporting/Render/Terminal.hs`
6. `make build && make test` — all existing tests pass

Phase 1: Error Codes & Registry (Week 2)

Goal: Assign codes to all existing errors.

1. Walk every constructor in `Syntax.Error`, `Canonicalize.Error`, `Type.Error`, `Pattern.Error`, `Import.Error`, `Main.Error`, `Docs.Error`
2. Assign stable `ErrorCode` to each
3. Create `Reporting/ErrorCode/Catalog.hs` mapping codes to short descriptions
4. Implement `canopy explain <code>` CLI command (reads from catalog)
5. Create initial documentation for top-20 most common errors
6. `make build && make test`

Phase 2: Diagnostic Conversion Layer (Week 3-4)

Goal: Each phase produces `Diagnostic` alongside existing `Report`.

1. Add `toDiagnostic` function to each `Reporting.Error.*` module
2. Wire `toDiagnostic` into `Reporting.Error.toReports` (produce both `Report` and `Diagnostic`)
3. Update JSON output to use `Diagnostic` when available
4. Keep terminal output using existing `Report` → `Doc` path (unchanged for now)
5. **Populate suggestions** for the most impactful error types:
   - `NotFoundVar` → edit-distance + import suggestions
   - `BadExpr` type mismatch → conversion function suggestions
   - `Incomplete` pattern → show missing cases as suggestion
   - `ModuleNameMismatch` → exact replacement suggestion
6. `make build && make test`

Phase 3: Multi-Span Rendering (Week 5)

Goal: Terminal output uses multi-span display.

1. Implement `renderMultiSpan :: Source -> [LabeledSpan] -> Doc` in `Reporting/Render/Code.hs`
2. Switch terminal renderer to use `Diagnostic` → multi-span rendering for type errors (biggest visual impact)
3. Gradually switch other phases
4. Add terminal width detection and adaptive formatting
5. `make build && make test`

Phase 4: Optimize & Generate Error Types (Week 6)

Goal: Fill the critical gap.

1. Create `Reporting/Error/Optimize.hs` with structured error ADT
2. Create `Reporting/Error/Generate.hs` with structured error ADT
3. Add `BadOptimize` and `BadGenerate` to `Reporting.Error`
4. Update `Optimize/Expression.hs`, `Optimize/Module.hs` to return `Either Optimize.Error` where failures are possible
5. Update `Generate/JavaScript.hs` to return `Either Generate.Error` for FFI linkage and other detectable failures
6. Add rendering functions following established patterns
7. `make build && make test`

Phase 5: Suggestion Engine (Week 7-8)

Goal: Smart, context-aware suggestions.

1. Implement import-aware suggestions: when a name isn't found, check if it exists in any installed package's exposed module
2. Implement type-aware field suggestions: for record field typos, prefer fields of compatible type
3. Implement "common JavaScript mistake" catalog for parse errors
4. Implement annotation-based suggestions for type errors: "your annotation says X but code produces Y, consider changing the annotation"
5. Add suggestion confidence ranking
6. Wire all suggestions through the `Diagnostic` pipeline
7. `make build && make test`

Phase 6: Cascade Prevention (Week 9)

Goal: Reduce error noise.

1. In canonicalization: if an import fails, mark all names from that import as "unavailable" rather than "not found" — produce ONE error ("import failed") instead of N errors ("name not found")
2. In type checking: track which errors are "primary" vs "secondary" (caused by a prior error) — suppress or de-prioritize secondary errors
3. Add error deduplication: if the same error appears at the same region, show it once
4. `make build && make test`

Phase 7: Golden Tests for Error Quality (Week 10)

Goal: Lock down error message quality.

1. Create `test/Golden/Errors/` directory
2. For each error code, create a `.can` source file that triggers it
3. Golden test captures the full terminal output

4. Golden test captures the full JSON output
5. Any error message change requires explicit golden file update
6. Aim for coverage of all error codes
7. `make build && make test`

Phase 8: Parse Error Recovery (Week 11-12)

Goal: Show multiple parse errors.

1. Add recovery points at top-level declaration boundaries
2. After a parse error, skip to the next `\n\n` or top-level keyword and continue parsing
3. Accumulate parse errors into a list instead of stopping at first
4. Update `BadSyntax Syntax.Error` to `BadSyntax (NE.List Syntax.Error)`
5. Render all parse errors with multi-module-style separators
6. `make build && make test`

Phase 9: LSP Integration (Week 13-14)

Goal: IDE-ready diagnostics.

1. Create `Reporting/Diagnostic/LSP.hs` that maps `Diagnostic` → LSP `Diagnostic` protocol
2. Include `CodeAction` for each `Suggestion` with `Definite` or `Likely` confidence
3. Include `DiagnosticRelatedInformation` for secondary spans
4. Wire into the canopy language server (if one exists) or create the foundation for one
5. `make build && make test`

PART 6 — DX VALIDATION

6.1 Metrics

Metric	Current (Estimated)	Target
Errors requiring external search	~40%	<10%
Errors with actionable suggestion	~30%	>90%
Errors with error code	0%	100%
Error message golden test coverage	0%	>95%
Cascade/noise errors shown	~20% of all errors	<5%
Parse errors shown per file	Max 1	Up to 5
JSON output contains suggestions	No	Yes
Multi-span diagnostics	2-span max	N-span

6.2 "Good Error" Heuristics

An error message is good if and only if:

1. **Locatable**: Points to the exact source location, not a surrounding context
2. **Understandable**: A developer with 1 month of Canopy experience can understand it without searching online
3. **Actionable**: Contains at least one concrete suggestion for how to fix it

4. **Non-cascading**: Is a primary error, not a downstream consequence of another error
5. **Stable**: Has a permanent error code that won't change across compiler versions
6. **Machine-readable**: JSON output contains all information needed for IDE integration

6.3 Before/After Examples

Example 1: Type Mismatch

BEFORE (current):

```
-- TYPE MISMATCH ---------------------------------------------- src/Main.can

The 2nd argument to `map` is not what I expect:

6|   List.map String.toInt names
                            ^^^^^
This `names` value is a:

    List String

But `map` needs the 2nd argument to be:

    List Int

Hint: I always figure out the argument types from left to right. If an
argument is acceptable, I assume it is correct and move on. So the problem
may actually be in one of the previous arguments!
```

AFTER (target):

```
-- TYPE MISMATCH [E0401] -------------------------------------- src/Main.can

The 2nd argument to `map` has type `List String`, but `List Int` is expected.

6|   List.map String.toInt names
                            ^^^^^
                            this is List String

3|   process : List Int -> List String
     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
     annotation expects List Int here

Try: If you want to convert strings to integers first, apply the conversion
before passing to this function:

    List.map String.toInt names
    → List.filterMap String.toInt names

Note: String.toInt returns Maybe Int, not Int. Use List.filterMap to handle
the Maybe, or provide a default with Maybe.withDefault.

Learn more: canopy explain E0401
```

Improvements: Error code, multi-span (annotation + usage), structured suggestion with replacement, confidence-rated note, learn-more link.

Example 2: Name Not Found

BEFORE (current):

```
-- NAMING ERROR ------------------------------------------------- src/Main.can

I cannot find a `String.fomrInt` variable:

8|   String.fomrInt 42
     ^^^^^^^^^^^^^^
These names seem close though:

    String.fromInt
    String.fromFloat
```

AFTER (target):

```
-- NAMING ERROR [E0301] ---------------------------------------- src/Main.can

I cannot find `String.fomrInt`.

8|   String.fomrInt 42
     ^^^^^^^^^^^^^^

Try: Did you mean `String.fromInt`?

    String.fomrInt → String.fromInt

Learn more: canopy explain E0301
```

Improvements: Error code, single best suggestion with replacement, less noise.

Example 3: Missing Pattern

BEFORE (current):

```
-- MISSING PATTERNS --------------------------------------------- src/Main.can

This `case` does not have branches for all possibilities:

10|>    case maybeUser of
11|>        Just user ->
12|>            process user

Missing possibilities include:

    Nothing

I would have to crash if I saw one of those. Add branches for them!
```

AFTER (target):

```
-- MISSING PATTERNS [E0501] ------------------------------------ src/Main.can

This `case` does not cover all possibilities.

10|   case maybeUser of
      ^^^^
11|       Just user ->
12|           process user

Missing: Nothing

Try: Add the missing branch:

    case maybeUser of
        Just user ->
            process user
+       Nothing ->
+           ...

Learn more: canopy explain E0501
```

Improvements: Error code, structured suggestion with exact code insertion, less verbose.

Example 4: Parse Error (cascading)

BEFORE (current — stops at first error):

```
-- UNFINISHED LET ---------------------------------------------- src/Main.can

I was partway through parsing a `let` expression, but I got stuck here:

5|   let x = 1
6|   y = 2
     ^
I was expecting to see the `in` keyword next.
```

AFTER (target — with recovery):

```
-- UNFINISHED LET [E0105] ------------------------------------- src/Main.can

I was expecting the `in` keyword to close this `let` expression:

5|   let x = 1
     ^^^
     this `let` needs a matching `in`

6|   y = 2
     ^
     unexpected start of expression

Try: Add `in` before the body expression:
```

```
    let x = 1
+   in
    y = 2


-- TYPE MISMATCH [E0401] ------------------------------------- src/Main.can

(continued parsing found another error...)

12|  foo True
         ^^^^
         ...
```

Improvements: Multi-error from parse recovery, labeled spans, structured suggestion.

Summary of Deliverables

Phase	Deliverable	Impact
0	Foundation types	Enables everything
1	Error codes + `canopy explain`	Learnability
2	Diagnostic conversion	JSON/LSP ready
3	Multi-span rendering	Visual quality
4	Optimize/Generate errors	Phase coverage
5	Suggestion engine	Actionability
6	Cascade prevention	Noise reduction
7	Golden tests	Quality regression prevention
8	Parse recovery	Multi-error for parse
9	LSP integration	IDE experience

Total estimated effort: ~14 weeks for a single engineer. Phases 0-4 are the highest ROI and should be prioritized. Phases 5-9 are incremental improvements that can be done over time.